

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

***CO1:** Apply object-oriented concepts and software development life cycle models to design structured and modular software systems. (BL2, BL3)*

Assessment Tool Weightage: 60%

Dr Gayathri A

QUESTIONS: 12

Total Marks:60

Annexure A

CONCEPT MAPPING

Code of Conduct:

*I **D Varshini** / 192320007 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

CONCEPT MAPPING

1. Construct a Concept Map

Given:

Requirement Analysis → _____ → Coding → Testing → _____ → Maintenance

(5 Marks)

Answer

Completed Concept Map

Requirement Analysis → Design → Coding → Testing → Deployment → Maintenance

Missing Concepts

1. **Design**
2. **Deployment**

Explanation

- **Requirement Analysis** identifies and documents the needs of users and the functions expected from the software.
- **Design** converts these requirements into a technical plan, including system architecture, modules, databases, interfaces, and workflows.
- **Coding** converts the design into an actual software program.
- **Testing** checks the software for errors and verifies whether it meets the specified requirements.
- **Deployment** releases the tested software into the actual working environment for end users.
- **Maintenance** involves correcting errors, improving performance, and updating the software after deployment.

Analysis

This sequence represents the major phases of the **Software Development Life Cycle (SDLC)**.

It:

- Ensures systematic and organised software development.
- Reduces project risks and development errors.
- Improves software quality and reliability.
- Makes the software easier to update and maintain.

- Ensures that each phase is completed properly before moving to the next phase.

Conclusion

The missing concepts are **Design** and **Deployment**. Design prepares the technical structure of the software, while deployment makes the completed software available to users.

2. Construct a Concept Map

Given:

Object-Oriented Principles → Abstraction → _____ → Inheritance → _____

(5 Marks)

Answer :

Completed Concept Map

Object-Oriented Principles → Abstraction → Encapsulation → Inheritance → Polymorphism

Missing Concepts

1. **Encapsulation**
2. **Polymorphism**

Explanation

- **Abstraction** hides unnecessary implementation details and displays only the essential features of an object.
- **Encapsulation** combines data and related methods within a single class. It also restricts direct access to sensitive data and protects it from unauthorised modification.
- **Inheritance** allows a new class to acquire the attributes and methods of an existing class, thereby reducing code duplication.
- **Polymorphism** allows the same method or interface to perform different behaviours depending on the object involved.

Analysis

Object-oriented principles:

- Improve data security through controlled access.
- Reduce software complexity by hiding unnecessary details.
- Enhance code reusability through inheritance.
- Increase flexibility by supporting different implementations.
- Reduce code duplication and development effort.
- Support scalable, organised, and maintainable software design.

Conclusion

The missing concepts are **Encapsulation** and **Polymorphism**. Encapsulation protects data, while polymorphism allows objects to respond differently to the same operation. Together, these principles improve the security, flexibility, and maintainability of software.

3. Construct a Concept Map

Given:

Class → Blueprint

Object → Instance

Method → _____

Attribute → _____

(5 Marks)

Answer:

Completed Concept Map

Class → Blueprint

Object → Instance

Method → Behavior

Attribute → State

Missing Concepts

1. **Behavior**
2. **State**

Explanation

- A class is a blueprint that defines the common properties and actions of objects.
- An object is an instance of a class with its own data.
- A method defines the behavior or action that an object can perform.
- An attribute represents the state, property, or characteristic of an object.

Analysis

These concepts:

- Help represent real-world entities in software.
- Clearly define the characteristics and actions of objects.
- Improve system organisation and modularity.
- Make software easier to understand, modify, and maintain.
- Support effective object-oriented analysis and design.

Conclusion

The missing concepts are Behavior and State. Methods describe what an object can do, while attributes describe the information or condition of the object.

4. A Concept Map Shows

Given:

Software Quality → Reliability → _____ → Maintainability

(5 Marks)

Answer :

Completed Concept Map

Software Quality → Reliability → Usability → Maintainability

Missing Concept

Usability

Explanation

- Software Quality refers to the ability of software to meet user requirements and perform its functions effectively.
- Reliability ensures that the software performs consistently without failure under specified conditions.
- Usability measures how easily users can learn, understand, and interact with the software.
- Maintainability refers to how easily the software can be corrected, modified, or updated after development.

Analysis

These software quality characteristics:

- Improve user satisfaction by making the software simple and reliable.
- Enhance product acceptance among users.
- Reduce software failures and operational problems.
- Make future updates, corrections, and modifications easier.
- Reduce maintenance time and cost.
- Contribute to the overall success and long-term use of the software.

Conclusion

The missing concept is Usability. Usability ensures that the software is easy to understand and operate, while reliability and maintainability improve its performance and long-term effectiveness.

5. UML Concept Map

Given:

UML

→ Structural Diagrams → _____

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → _____

Answer:

Completed Concept Map:

UML

→ Structural Diagrams → Object Diagram

→ Behavioral Diagrams → Activity Diagram

→ Interaction Diagrams → Communication Diagram

Missing Concepts

1. Object Diagram
2. Communication Diagram

Explanation

- An Object Diagram represents object instances, their attributes, and the relationships between them at a specific point in time.
- A Communication Diagram represents the exchange of messages and interactions among objects to perform a particular function.

Analysis

UML diagrams:

- Improve the visual representation of a software system.
 - Support object-oriented analysis and design.
 - Help developers understand system structure and behaviour.
 - Facilitate clear communication among developers and stakeholders.
 - Improve system documentation and maintainability.
-

6. Analyse the software development process

Given :

Requirements ✓ → Design ✓ → Coding ✗ → Testing ✗ → Deployment ✗

Answers :

Identification of Failure

The failure occurs during the Coding phase. Although the requirements and design phases are completed successfully, errors in program logic or incorrect implementation prevent the software from progressing to the testing and deployment phases.

Explanation

- The coding phase converts the system design into an executable software program.
- Implementation errors, syntax issues, or incorrect program logic may cause coding failure.
- Since the software is not implemented correctly, testing cannot be carried out effectively.
- Deployment is also delayed because only properly tested software can be released to users.

Analysis

- Errors in coding affect all subsequent phases of development.
- Testing may produce inaccurate results or may not begin at all.

- Project completion is delayed.
- Development costs and rework increase.
- Overall software quality and reliability decrease.

Solution

- Review and debug the source code carefully.
 - Follow standard coding practices and naming conventions.
 - Conduct peer reviews and code inspections.
 - Perform unit testing before complete system testing.
 - Correct implementation errors before proceeding to the testing phase.
-

7. Construct a Concept Map

Given:

Problem Identification → _____ → Design → Implementation → _____

Answer:

Completed Concept Map

Problem Identification → Requirement Analysis → Design → Implementation → Testing

Missing Concepts

1. **Requirement Analysis**
2. **Testing**

Explanation

1. Problem Identification

Problem identification is the first stage of software development. In this stage, the actual problem faced by the user or organisation is clearly recognised and defined.

It helps the development team understand:

- The existing problem or limitation.
- The purpose of developing the software.
- The expected outcome of the proposed system.
- The users who will interact with the system.

A clearly identified problem provides a strong foundation for the remaining development activities.

2. Requirement Analysis

Requirement Analysis is the process of collecting, studying, and documenting the needs of users and stakeholders.

During this phase, the development team identifies:

- Functional requirements, such as the operations the system must perform.
- Non-functional requirements, such as security, reliability, usability, and performance.
- User expectations and business objectives.
- System limitations and technical constraints.

For example, in an online shopping system, requirements may include product searching, user registration, online payment, order tracking, and secure data handling.

Proper requirement analysis ensures that the development team clearly understands what must be created before starting the design process.

3. Design

The design phase converts the identified requirements into a structured plan for developing the software.

It includes:

- Designing the overall system architecture.
- Dividing the system into different modules.
- Designing the database and user interface.
- Defining the relationship between system components.
- Planning how data will move through the system.

A good system design makes implementation easier and reduces confusion during development.

4. Implementation

Implementation is the phase in which the system design is converted into a working software application.

During this phase:

- Developers write the required programs.
- Individual modules are created.
- Databases and interfaces are developed.
- Different system components are integrated.
- Coding standards and development guidelines are followed.

The success of implementation depends heavily on the accuracy of the requirements and design.

5. Testing

Testing is the process of checking whether the implemented software works correctly and meets the specified requirements.

Testing helps identify:

- Coding errors.
- Incorrect outputs.
- Missing functions.
- Security problems.
- Performance issues.
- Integration problems between modules.

Different testing methods, such as unit testing, integration testing, system testing, and acceptance testing, may be performed to verify the software.

Testing ensures that the final system is reliable, accurate, secure, and ready for use.

Analysis

The sequence of these stages provides a systematic approach to software development.

- **Improves software quality:** Each phase is completed and verified before moving to the next phase.
- **Reduces development errors:** Clear requirements and proper testing help identify mistakes at an early stage.
- **Prevents misunderstanding:** Requirement analysis ensures that developers and users have a common understanding of the system.
- **Reduces rework and cost:** Early identification of requirements and errors prevents expensive corrections later.
- **Improves reliability:** Testing confirms that the software performs consistently under different conditions.
- **Ensures user satisfaction:** The final system is developed according to actual user needs.
- **Supports successful project completion:** A structured development process helps complete the project within the planned time, cost, and quality limits.

Conclusion

The completed concept map represents the major stages involved in developing a software solution. Requirement analysis connects the identified problem with the system design, while testing verifies the correctness of the implemented system. Therefore, both phases are essential for creating reliable, maintainable, and high-quality software.

8. Construct a Concept Map

Given:

Object-Oriented Features

→ Class

→ _____

→ Message Passing

→ _____

Answer:**Completed Concept Map**

Object-Oriented Features → Class → Object → Message Passing → Dynamic Binding

Missing Concepts

1. **Object**
2. **Dynamic Binding**

Explanation**1. Class**

A **class** is a blueprint or template used to create objects. It defines the common attributes and methods that objects of the class will possess.

For example, a Student class may contain attributes such as name, register number, and department, along with behaviours such as registering for a course or viewing results.

2. Object

An **object** is a real instance of a class. It contains its own data and can perform the behaviours defined by its class.

For example, if Student is a class, a particular student with a specific name and register number is an object of that class.

Objects help represent real-world entities in a software system.

3. Message Passing

Message passing is the process through which objects communicate with one another. One object sends a request or invokes a method of another object to perform a particular task.

For example, an order object may send a message to a payment object to process an online transaction.

Message passing supports interaction between different parts of an object-oriented system without exposing their internal implementation.

4. Dynamic Binding

Dynamic binding refers to resolving a method call at runtime rather than during compilation. The method that is executed depends on the actual object associated with the method call. It is commonly used when a parent-class reference refers to an object of a child class. The overridden method of the child class is selected during program execution.

Dynamic binding is an important mechanism for achieving **runtime polymorphism**.

Analysis

Object-oriented features provide several benefits:

- **Supports design flexibility:** Different objects can respond differently to the same method call.
- **Enables runtime polymorphism:** The appropriate method is selected according to the actual object at runtime.
- **Improves maintainability:** Existing classes can be modified or extended with minimal impact on other components.
- **Promotes modularity:** The system is divided into independent objects with specific responsibilities.
- **Improves reusability:** Classes and objects can be reused in different parts of the application.
- **Reduces system complexity:** Message passing provides a structured way for objects to communicate.
- **Supports scalability:** New classes and behaviours can be added as the software grows.

Conclusion

Objects are instances of classes that represent real-world entities, while message passing enables communication between them. Dynamic binding determines the appropriate method during runtime and supports runtime polymorphism. Together, these features make object-oriented software flexible, reusable, and maintainable.

9. Construct a Concept Map

Given:

Software Development Models → Waterfall → _____ → Spiral → _____

Answer:

Completed Concept Map

Software Development Models → Waterfall → Incremental Model → Spiral → Agile Model

Missing Concepts

1. **Incremental Model**
2. **Agile Model**

Explanation

1. Waterfall Model

The **Waterfall Model** follows a sequential approach in which software development progresses through clearly defined phases such as requirement analysis, design, coding, testing, deployment, and maintenance.

Each phase is generally completed before the next phase begins. This model is suitable when the project requirements are clear, stable, and unlikely to change.

2. Incremental Model

The **Incremental Model** develops and delivers software in multiple smaller stages called increments.

Each increment provides a working portion of the complete system. Additional features are gradually added in later increments until the full software product is completed.

For example, an online shopping application may initially provide user registration and product searching. Payment, order tracking, and customer support features may be added in later increments.

The Incremental Model:

- Provides usable software at an early stage.
- Makes testing and debugging easier.
- Allows customer feedback after each release.
- Reduces the impact of development errors.
- Supports the gradual addition of new features.

3. Spiral Model

The **Spiral Model** combines iterative development with systematic risk analysis. Software is developed through repeated cycles known as spirals.

Each cycle generally includes:

- Planning and requirement identification.
- Risk analysis.
- Design and development.
- Testing and customer evaluation.

The Spiral Model is suitable for large, complex, and high-risk projects because risks are identified and managed during every development cycle.

4. Agile Model

The **Agile Model** follows an iterative and flexible approach in which software is developed through short development cycles called iterations or sprints.

Developers, testers, customers, and stakeholders work closely throughout the project. Customer feedback is collected continuously and used to improve the software in subsequent iterations.

The Agile Model:

- Responds quickly to changing requirements.
- Encourages regular customer involvement.
- Delivers working software frequently.
- Improves collaboration among team members.
- Supports continuous testing and improvement.

Analysis

Software development models provide different methods for planning, developing, testing, and delivering software.

- **Enhances adaptability:** Incremental and Agile models allow changes to be introduced during development.
- **Reduces project risks:** The Spiral Model identifies and evaluates risks in every development cycle.
- **Improves customer satisfaction:** Customers can review working software and provide feedback regularly.
- **Supports early delivery:** The Incremental and Agile models deliver functional parts of the software before the entire system is completed.
- **Improves software quality:** Frequent testing helps identify and correct errors at an early stage.
- **Simplifies project management:** Dividing the project into phases or iterations makes progress easier to monitor.
- **Supports different project requirements:** Organisations can select an appropriate model depending on project size, risk, complexity, and requirement stability.

Conclusion

The Incremental Model delivers software in manageable stages, while the Agile Model supports iterative development and continuous customer feedback. Along with Waterfall and Spiral, these models provide structured approaches for developing reliable and high-quality software. Selecting the appropriate development model improves adaptability, reduces project risks, and increases customer satisfaction.

10. Construct a Concept Map

Given:

Class Diagram → **Class** → _____ → **Association** → _____

Answer :

Completed Concept Map

Class Diagram → **Class** → **Attribute** → **Association** → **Method**

Missing Concepts

1. Attribute
2. Method

Explanation

1. Class Diagram

A Class Diagram is a structural UML diagram that represents the static structure of an object-oriented system. It shows the classes present in the system, their attributes, methods, and relationships.

Class diagrams are commonly used during the analysis and design phases to understand how different components of a software system are organised.

2. Class

A class is a blueprint or template used to create objects. It groups related data and operations into a single unit.

For example, a Student class may contain information such as student name, register number, and department, along with operations such as enrolling in a course or viewing results.

In a UML class diagram, a class is usually represented using a rectangle divided into three sections:

- Class name
- Attributes
- Methods

3. Attribute

An attribute defines a property, characteristic, or data value associated with a class. It represents the state or information maintained by the objects created from that class.

For example, the attributes of a Student class may include:

- Name

- Register number
- Department
- Email address

Each student object can have different values for these attributes. Therefore, attributes help distinguish one object from another.

4. Association

An association represents a relationship or connection between two or more classes. It shows how objects of one class interact with or are related to objects of another class.

For example:

- A student enrolls in a course.
- A doctor treats a patient.
- A customer places an order.

Associations may also indicate multiplicity, such as one-to-one, one-to-many, or many-to-many relationships.

5. Method

A method defines an operation, behaviour, or function that can be performed by a class or its objects.

For example, the methods of a Student class may include:

- Register for a course
- Pay fees
- View attendance
- Download results

Methods describe what an object can do, while attributes describe the information an object contains.

Analysis

Class diagrams provide several benefits in object-oriented software development:

- Represent object structure effectively: They clearly show classes, attributes, methods, and relationships.
- Improve software documentation: They provide a visual reference that can be used throughout the development process.
- Support object-oriented design: They help developers convert system requirements into organised software classes.
- Improve communication: Developers, designers, and stakeholders can use the diagram to understand the system structure.

- Promote maintainability: Changes to classes and relationships can be identified and managed easily.
- Support reusability: Well-designed classes can be reused in different modules or software projects.
- Reduce development errors: Identifying attributes, methods, and relationships before coding prevents design mistakes.
- Simplify implementation: Developers can use the class diagram as a blueprint while writing the program.

Conclusion

Attributes define the data or properties maintained by a class, while methods define the operations performed by it. Associations describe the relationships between classes. Together, these elements allow a class diagram to represent the static structure of an object-oriented system clearly and effectively.

11. Construct a Concept Map

Given:

Software Engineering Principles → Modularity → _____ → Reusability → _____

Answers :

Completed Concept Map

Software Engineering Principles → Modularity → Maintainability → Reusability → Scalability

Missing Concepts

1. Maintainability
2. Scalability

Explanation

1. Software Engineering Principles

Software engineering principles are systematic guidelines used to design, develop, test, deploy, and maintain reliable software systems.

These principles help developers create software that is:

- Well organised
- Easy to understand
- Reliable

- Adaptable to change
- Capable of supporting future requirements

Applying proper software engineering principles reduces complexity and improves the overall quality of the software.

2. Modularity

Modularity is the process of dividing a large software system into smaller and independent units called modules.

Each module is designed to perform a specific function. For example, an e-commerce system may contain separate modules for:

- User registration
- Product management
- Payment processing
- Order tracking
- Customer support

Modularity makes software easier to design, develop, test, and debug. A change made in one module generally has minimal impact on other modules.

3. Maintainability

Maintainability refers to how easily software can be corrected, updated, improved, or adapted after its development.

Maintainable software has:

- Clear and understandable structure
- Proper documentation
- Consistent coding standards
- Independent modules
- Minimum duplication
- Well-defined interfaces

For example, when a new payment method is introduced, maintainable software allows developers to add it without redesigning the entire system.

Maintainability is important because software frequently requires bug fixes, security updates, feature improvements, and compatibility changes.

4. Reusability

Reusability is the ability to use existing software components, classes, modules, or functions in different parts of the same system or in other projects.

For example, a login module developed for one application may be reused in another application with only minor modifications.

Reusability:

- Reduces repeated development work
- Saves development time and cost
- Improves consistency
- Minimises coding errors
- Increases developer productivity

Reusable components are generally well designed, independent, and properly documented.

5. Scalability

Scalability is the ability of a software system to handle an increasing number of users, transactions, data, or operations without a significant reduction in performance.

For example, an online shopping system may initially support one thousand users. As the business grows, the system should be capable of supporting several lakh users without frequent failures or delays.

Scalability can be achieved by:

- Improving system architecture
- Adding computing resources
- Optimising databases
- Distributing workloads
- Using independent and reusable modules

A scalable system can expand according to future business and user requirements.

Analysis

The principles of modularity, maintainability, reusability, and scalability provide several advantages:

- Improves software performance: A modular and scalable architecture distributes tasks efficiently and prevents system overload.
- Reduces maintenance effort: Independent modules make it easier to identify and correct errors.
- Supports long-term software evolution: Maintainable software can be modified as technology and user requirements change.
- Reduces development cost: Reusable components avoid the need to develop the same functionality repeatedly.
- Improves reliability: Previously tested components can be reused with fewer errors.

- Supports business growth: Scalable software can accommodate more users, data, and transactions.
- Simplifies testing: Individual modules can be tested separately before complete system integration.
- Improves flexibility: New modules and features can be added without changing the entire system.
- Enhances software quality: A properly structured system is easier to understand, test, and manage.

Conclusion

Maintainability enables software to be corrected and updated easily, while scalability allows it to manage increased workloads efficiently. Along with modularity and reusability, these principles produce software that is flexible, reliable, cost-effective, and capable of supporting long-term growth.

12. Given:

Requirement Analysis ✓ → System Design ✓ → Coding ✓ → Integration ✗ → Deployment ✗

Answer :

Identification of Failure

The failure occurs during the **Integration phase**. Requirement analysis, system design, and coding have been completed successfully, but the separately developed modules fail to work together as a complete system.

Because integration is unsuccessful, the software cannot proceed to the deployment phase.

Explanation

1. Requirement Analysis

In this phase, user needs, system functions, performance expectations, and technical requirements are identified and documented.

Since this phase is marked as successful, the development team has clearly understood what the software must accomplish.

2. System Design

The system design phase converts the requirements into a technical structure. It defines:

- System architecture
- Modules and components
- Database structure
- User interfaces
- Communication between modules

The successful completion of this phase indicates that the overall software structure has been planned properly.

3. Coding

During coding, developers convert the system design into working program modules.

Since coding is completed successfully, the individual modules may function correctly when tested separately. However, successful coding does not guarantee that all modules will communicate correctly when combined.

4. Integration

Integration is the process of combining independently developed modules into a complete software system.

For example, in an online shopping system:

- The product module must communicate with the cart module.
- The cart module must send details to the payment module.
- The payment module must update the order and inventory modules.

Integration failure may occur because of:

- Incorrect module interfaces
- Incompatible data formats
- Communication errors
- Different software versions
- Incorrect parameter passing
- Database connection problems
- Missing dependencies
- Conflicting business logic

Even when individual modules operate correctly, the complete system may fail if the modules do not exchange data or messages properly.

5. Deployment

Deployment involves releasing the tested software into the actual user environment.

Since the modules are not properly integrated, deployment cannot proceed. Releasing incompletely integrated software may cause system crashes, incorrect outputs, data loss, or security problems.

Analysis

Integration failure can create several problems:

- **Causes functional inconsistencies:** Different modules may produce conflicting or incorrect results.
- **Delays software deployment:** The system cannot be released until integration problems are corrected.
- **Increases debugging effort:** Developers must examine several modules to locate the exact source of failure.
- **Raises development cost:** Additional time, tools, and resources are required to resolve compatibility issues.
- **Affects software reliability:** The system may fail while exchanging information between modules.
- **Creates data inconsistencies:** Data generated by one module may not be accepted or processed correctly by another module.
- **Reduces user confidence:** A poorly integrated system may provide incomplete or incorrect services.
- **Affects project schedules:** Delays in integration may postpone testing, deployment, and final delivery.

Solution

The following corrective measures should be taken:

1. **Perform integration testing :**Combine the modules gradually and test whether they work correctly as a complete system.
2. **Verify module interfaces :**Check the input, output, parameters, communication protocols, and data structures used by each module.
3. **Resolve compatibility issues :** Ensure that all modules use compatible software versions, libraries, databases, and data formats.
4. **Use incremental integration :** Integrate and test a few modules at a time instead of combining the entire system at once.
5. **Check data flow:** Verify that data is transferred accurately between different modules without loss or modification.

6. **Review interface documentation** :Maintain clear documentation explaining how each module communicates with other components.
7. **Conduct regression testing** : After correcting integration issues, retest the previously working functions to ensure that the changes have not introduced new errors.
8. **Use debugging and monitoring tools** : Examine system logs, error messages, and communication records to locate integration problems.

Conclusion

The failure occurs during the integration phase because the individual software modules cannot communicate or function together correctly. Integration testing, interface verification, compatibility checking, and gradual module integration must be carried out before deployment. Resolving these issues ensures that the complete software system is reliable, consistent, and ready for release.

RUBRICS FOR CALCULATION

CONCEPT MAPPING

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand